Milan Restaurant Ratings Across Online Platforms

Data Management Project

Edoardo Zero
Matricola: 947907
e.zero1@campus.unimib.it

Simone Ragusini
Matricola: 945119
s.ragusini@campus.unimib.it

Vladyslav Yelisieiev
Matricola: 947134
v.yelisieiev@campus.unimib.it

University of Milan-Bicocca Academic Year 2025/2026

Abstract

Online review platforms are a primary source of information for consumers choosing where to eat, yet the same restaurant is often rated differently on each of them. This project builds an integrated database of restaurants in Milan from three platforms — Google Maps, Tripadvisor, and TheFork — and uses it to study how consistent restaurant ratings and metadata are across sources.

A complete ELT pipeline was developed. Data are acquired through the Google Places API and two dedicated web scrapers, loaded as raw documents into MongoDB, cleaned and normalized per source, and then integrated. Integration relies on schema matching, a Google-anchored entity-resolution stage with calibrated thresholds, dedicated handling of chain and franchise venues, and an optional large-language-model adjudication of the uncertain cases. The resulting Google-seeded collection preserves each platform's ratings side by side and is projected into a flat ClickHouse layer for analytical querying. The quality of the source data is assessed before integration, while the entity-resolution process that drives the integration is evaluated afterwards against a manually labelled gold standard.

The final dataset contains 10,054 restaurants. Ratings are generally consistent across platforms, but Tripadvisor is systematically more severe than Google and TheFork; rating level and cross-platform agreement vary by cuisine, with mainstream categories more contested than specialist ones; and data completeness is strongly spatially structured, decreasing from the historic centre towards the periphery.

1 Introduction

Online review platforms have become a key source of information for consumers when choosing restaurants. However, ratings and restaurant information may vary considerably across platforms due to differences in user behavior, data collection processes and data quality. Understanding these discrepancies is important for assessing the reliability of online restaurant evaluations.

This project investigates restaurant data collected from three major platforms: Google Maps, Tripadvisor and TheFork [2, 3, 4]. The analysis focuses on restaurants located in Milan, with the objective of building an integrated database suitable for comparative analysis.

To support this goal, a complete ELT (Extract, Load, Transform) pipeline was developed. Data were extracted from the three sources, loaded into MongoDB, and then cleaned, enriched, transformed, and integrated. MongoDB is used as the document-oriented system of record for raw, clean, and integrated collections, while ClickHouse materializes the analytical layer for the research-question queries [6, 7]. Docker ensures reproducible and portable infrastructure [8]. The implementation uses Python managed with `uv`, Pydantic for validation, PyMongo and ClickHouse Connect for database access, and pandas, SciPy, and Plotly for analysis outputs [9, 14, 12, 13]. The source code is publicly available in the GitHub repository [1].

1.1 Project Components

The project is composed of six main components, each corresponding to one stage of the pipeline and to a distinct set of services in the repository:

- **Data acquisition:** a Google Places API extractor that builds the Milan seed list, and two dedicated Playwright web scrapers for Tripadvisor and TheFork.
- **Storage and loading:** a generic loader that ingests the raw files into MongoDB, the document-oriented system of record.
- **Cleaning and transformation:** per-platform normalization, spatial enrichment through geocoding, and schema matching that align the three sources to a common representation.
- **Entity resolution and integration:** a Google-anchored record-linkage stage, with optional LLM adjudication of uncertain pairs, that produces the unified integrated dataset.
- **Quality assessment:** pre-integration profiling of each source and post-integration evaluation of the entity-resolution results against a manually labelled gold standard.

- **Analysis:** a ClickHouse analytical layer and a set of notebooks that answer the research questions and generate the report's tables and figures.

The overall software architecture that ties these components together is described in the next chapter. The study focuses on the following three research questions:

- **RQ1.** Are some platforms systematically optimistic or pessimistic in the ratings they assign?
- **RQ2.** Does the rating level and the cross-platform consistency of ratings vary by cuisine?
- **RQ3.** Does the geographic location of a restaurant affect the completeness of its data?

The integrated dataset supports further analyses beyond these three questions — for example, the effect of review volume on rating stability, the role of price level, the impact of multi-platform presence, and the relationship between photo activity and engagement. These additional questions are explored as part of the data-visualization module of the course and are left as future work.

2 Software Architecture

The project is implemented as a modular, containerized data pipeline built around the ELT (Extract, Load, Transform) paradigm. Rather than a single monolithic program, the system is decomposed into a sequence of independent stage services that communicate exclusively through shared database collections. Each stage reads the output that the previous stage persisted, applies its own responsibility, and writes a new collection, so that the stages remain loosely coupled and individually re-runnable.

2.1 Pipeline Organization

The codebase is organized by pipeline stage rather than by data source. Services live under `services/extract`, `services/load`, and `services/transform`, and are exposed as PEP 420 namespace packages so that each import path reflects its stage (for example `extract.google_places_api`, `load.mongo`, and `transform.entity_resolution`). Every stage is invoked through a dedicated command-line entry point, which keeps the boundaries between acquisition, loading, cleaning, integration, quality assessment, and analysis explicit and reproducible. This separation is a deliberate design constraint: no functionality is added as a standalone script, and each component belongs to exactly one stage.

The end-to-end flow follows six sequential stages: (i) data acquisition, (ii) loading into the system of record, (iii) per-platform cleaning and transformation, (iv) entity resolution and integration, (v) quality assessment, and (vi) analytical querying. Figure 1 gives an overview of the resulting architecture, showing how the stage services communicate through raw files, MongoDB collections, ClickHouse tables, and report artifacts. The following chapters describe each stage in detail.

2.2 Storage Architecture

Persistence follows a two-layer, polyglot design. MongoDB is the document-oriented system of record: it stores the raw source documents, the cleaned per-platform collections, and the final integrated dataset, preserving heterogeneous structures without imposing a rigid schema. ClickHouse is added downstream as a column-oriented analytical store, holding flat projections of the cleaned and integrated collections that are optimized for the aggregations, comparisons, and geographic grouping required by the research questions. This split lets each engine serve the workload it is best suited for, while keeping the analytical layer decoupled from the operational store. Both databases run as reproducible Docker services, so the whole infrastructure can be recreated identically across the team's machines. The storage and integration chapters detail the two layers.

2.3 Why ELT

The pipeline loads the raw data into MongoDB before any transformation, in contrast to a traditional ETL flow that would clean and reshape the data before storage. Loading first preserves each source in its original form, which is essential for a project whose goal is to compare and audit cross-platform discrepancies: the untouched documents remain available for provenance, reprocessing, and quality assessment. Transformations are then applied inside the storage layer in controlled, versioned stages, so that any cleaning or integration decision can be revisited without re-acquiring the data.

2.4 Technology Stack

Table 1 summarizes the technologies adopted and the role each one plays. The stack was chosen to favour reproducibility and team collaboration: a single Python version pinned through `uv`, containerized databases, and a shared repository with code review.

Concern	Technology and role
Language & environment	Python 3.12 and uv for implementation and reproducible dependency management.
Data acquisition	Google Places API, Playwright, and Chromium for API seed extraction and browser-based scraping.
Validation	Pydantic for configuration and data validation.
System of record	MongoDB 7 for raw, clean, and integrated document collections.
Analytical store	ClickHouse for flat tables and research-question queries.
Infrastructure	Docker for reproducible database services.
Integration	RapidFuzz, GeoPy, and Nominatim for fuzzy matching and geocoding enrichment.
LLM adjudication	OpenAI for optional labelling of uncertain entity-resolution pairs.
Analysis	pandas, SciPy, Plotly, and Jupyter for analysis, statistics, and visual outputs.
Collaboration	GitHub, OpenAI, and Claude for version control, code review, and AI-assisted development.

Table 1: Technology stack and the role of each tool in the software architecture.

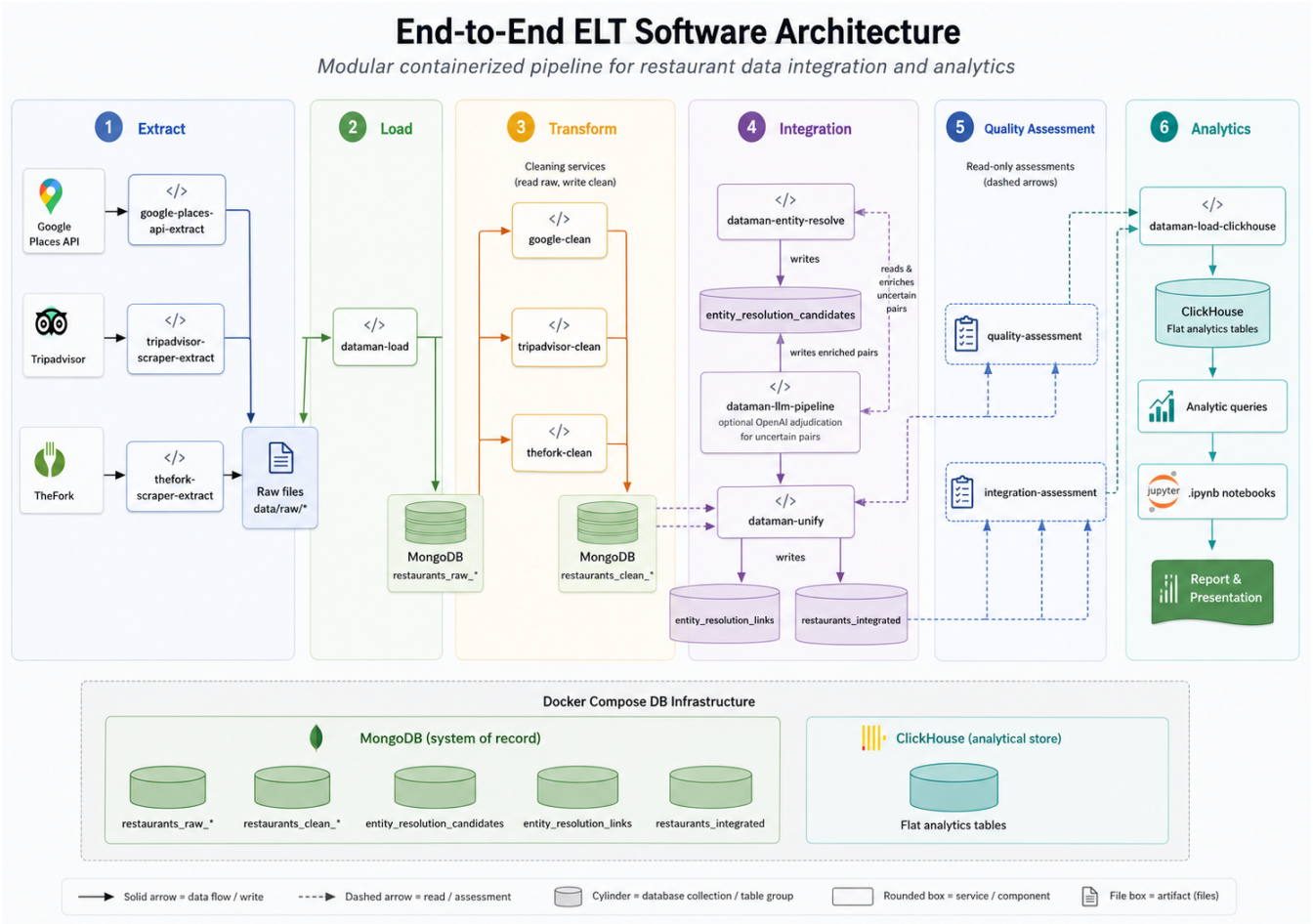


Figure 1: End-to-end ELT software architecture. Solid arrows represent data flow and materialized writes, while dashed arrows represent read-only assessment or query access. MongoDB acts as the system of record for raw, clean, entity-resolution, and integrated collections; ClickHouse provides the flat analytical tables used by the notebooks and final report.

3 Data Acquisition

The raw data acquisition phase was structured across three distinct data sources, leveraging an Application Programming Interface (API) and two differentiated web scraping methodologies. Specifically, three industry-leading platforms predominantly utilized for restaurant evaluation and selection were targeted: Google, TripAdvisor and TheFork. The subsequent sections delineate the technical frameworks and specific acquisition methodologies deployed for each platform.

3.1 Google Places - API

The objective of this acquisition phase was to build a comprehensive repository of restaurants located in Milan, which would subsequently serve as the reference dataset for cross-platform integration.

Due to the limitations imposed by the Google Places API [2], namely the restriction to circular search areas and the maximum number of results returned per request, a dedicated acquisition strategy was developed. The territory was divided into partially overlapping search regions (*tiles*), each generating an independent Nearby Search request. Since dense urban areas can saturate API results and hide additional venues, the seed was collected through the progressive multi-pass strategy summarized in Table 2, rather than through a single city-wide crawl.

To further improve coverage in highly dense areas, a multi-centre strategy was adopted by defining additional neighbourhood anchors in districts characterized by a high concentration of restaurants, such as Navigli, Brera, Porta Venezia, and Porta Romana (Table 3).

To ensure efficiency and reproducibility, overlapping search results were automatically deduplicated using the unique Google `place_id`, while a checkpoint mechanism allowed interrupted executions to be resumed without repeating completed requests.

The acquisition process produced a raw dataset containing 10,808 unique restaurants. For each venue, core information such as identifier, name, address, geographic coordinates, categories, rating, and review count was collected. The dataset was subsequently enriched through the Place Details endpoint, adding supplementary attributes including contact information, opening hours, pricing details, service options, accessibility features, photographs, and user reviews. Owing to its complete geographic coverage and rich metadata, the resulting dataset was adopted as the primary reference source for the subsequent integration and analysis stages.

Pass	Tile radius	Area	Purpose
1	750 m	Whole-city search around Milan	Broad coverage of the Milan restaurant universe.
2	300 m	Whole-city refinement	Denser tiling to recover venues missed because of result saturation.
3	100 m	Dense neighbourhood anchors	Fine-grained recovery in high-density restaurant districts.

Table 2: Progressive Google Places acquisition strategy used to improve spatial coverage and reduce API saturation effects.

Neighbourhood	Anchor	Center Coordinates	Outer Radius
Duomo	duomo	(45.4642, 9.1900)	1200 m
Navigli	navigli_1	(45.4520, 9.1760)	600 m
Navigli	navigli_2	(45.4485, 9.1720)	600 m
Navigli	navigli_3	(45.4450, 9.1680)	600 m
Brera	brera	(45.4720, 9.1880)	600 m
Isola	isola	(45.4870, 9.1880)	600 m
Porta Venezia	porta_venezia_1	(45.4740, 9.2050)	600 m
Porta Venezia	porta_venezia_2	(45.4790, 9.2110)	600 m
Porta Romana	porta_romana_1	(45.4540, 9.2010)	600 m
Porta Romana	porta_romana_2	(45.4490, 9.2050)	600 m
Sempione	sempione_1	(45.4790, 9.1740)	600 m
Sempione	sempione_2	(45.4830, 9.1680)	600 m
Sempione	sempione_3	(45.4870, 9.1620)	600 m
Loreto	loreto	(45.4860, 9.2160)	600 m

Table 3: Neighbourhood anchors used to increase restaurant coverage in high-density areas of Milan.

3.2 TripAdvisor - Web Scraping

Since Tripadvisor does not provide a public API suitable for large-scale data extraction, a dedicated web scraping pipeline was developed to collect restaurant information directly from the platform. Tripadvisor was selected as a complementary source because it represents one of the largest restaurant-review platforms and provides valuable user-generated content that can be compared with ratings from other sources.

The acquisition process was implemented using Python and Playwright [10]. Given Tripadvisor's anti-automation mechanisms, including browser fingerprinting and behavioural analysis, the scraper relied on Playwright's *launch_persistent_context()* functionality. This approach maintains browser cookies and session data across executions, allowing the scraper to emulate a real user and reducing the probability of being blocked.

The workflow was organized into two phases. The first phase explored restaurant listing pages within Milan and collected restaurant profile URLs, while the second phase visited each profile page to extract detailed information. Checkpoint files were used throughout the process to ensure fault tolerance and allow interrupted executions to resume without repeating completed operations.

The acquisition process generated 7,539 unique restaurant records stored in JSON format. The collected information includes restaurant identification data, ratings, review counts, cuisine categories, opening hours, price ranges, and user reviews. Unlike the other sources, the raw dataset preserves values exactly as displayed on Tripadvisor, postponing normalization and type conversion to the subsequent transformation stage. For example, ratings are stored with Italian comma decimals, review counts may contain thousands separators and textual wrappers, and price bands are represented using source-specific symbols.

The Tripadvisor scraper does not obtain geospatial attributes natively like Google Places, but it offers a considerably richer collection of user reviews. For this reason, it represents an important source for analysing restaurant reputation and investigating rating consistency across platforms. Geographic enrichment is therefore deferred to the Tripadvisor cleaning stage, where addresses are geocoded before integration.

3.3 TheFork - Web Scraping

TheFork was selected as the third data source due to its specialization in restaurant reservations and its widespread adoption within the European restaurant industry. Since no public API is available for large-scale extraction, a dedicated web scraping pipeline was developed in Python.

The acquisition process followed a two-stage workflow. First, restaurant listing pages were explored to identify venues operating within Milan. Subsequently, each restaurant page was visited to collect detailed information including ratings, cuisine categories, prices, discounts,

opening hours, contact information, geospatial data, photographs, and customer reviews.

A distinctive characteristic of the TheFork pipeline is the integration of data normalization directly within the acquisition process. Unlike Tripadvisor, where the extracted values are preserved in their original form, TheFork records are standardized immediately after extraction. Ratings, review counts, URLs, and structured attributes are converted into a consistent format, producing a dataset that is already aligned with the integration requirements of the project.

Given the anti-automation mechanisms adopted by TheFork, the scraper incorporates a proxy-rotation strategy to mitigate blocking events and HTTP 403 responses. Requests can be distributed across multiple proxy servers, improving scalability and allowing the acquisition process to continue even when individual endpoints become temporarily unavailable.

An additional resilience mechanism was implemented through Playwright and the Chrome DevTools Protocol (CDP). Instead of relying exclusively on traditional browser navigation, the scraper can interact with TheFork's internal GraphQL services through an authenticated browser session. This approach reduces the risk of triggering anti-bot protections and improves the recovery of missing information from restaurant profiles.

The acquisition framework also supports parallel and distributed execution, allowing multiple browser instances or machines to process different subsets of restaurants simultaneously. This significantly reduces collection time while preserving consistency across the generated records.

The resulting dataset adopts a normalized schema specifically designed to facilitate integration with Google Places and Tripadvisor. By combining automated normalization, validation procedures, and enriched restaurant metadata, TheFork provides a highly structured source that requires less type repair than Tripadvisor, while still passing through a dedicated cleaning stage to standardize prices, cuisines, discounts, opening hours, reviews, and quality flags.

4 Data Storage and Loading

This chapter describes the process through which the extracted files were loaded into MongoDB and organized into separate raw collections.

The loading phase was designed as part of an ELT pipeline, this architectural choice allows the original data to be preserved as faithfully as possible, avoiding early transformations that could alter the raw information collected from the different sources.

The storage architecture is supported by a Docker-based infrastructure [8], which ensures that the database environment can be reproduced consistently across different machines. This is particularly important in a collaborative project, where multiple contributors may execute the same pipeline components on different operating systems and configurations.

4.1 Raw Data Loading into MongoDB

MongoDB was adopted as the storage system for the raw datasets [6] because the project integrates information collected from heterogeneous sources with different structures, formats, and levels of completeness. Google Places, Tripadvisor and TheFork do not expose the same attributes and do not organize restaurant information according to a shared schema. A document-oriented database is therefore appropriate because it allows each restaurant record to be stored as a flexible JSON-like document without requiring a rigid relational schema at the loading stage.

This choice is coherent with the objectives of the project. Since the analysis aims to compare ratings, metadata quality, and platform-specific discrepancies, it is important to preserve the original structure of each source before applying cleaning, integration and normalization procedures. MongoDB acts as the document-oriented system of record, storing the extracted datasets in their original form and later hosting the clean and integrated collections. The analytical SQL layer is introduced downstream through ClickHouse [7], after the integrated dataset has been materialized.

The loading layer follows an ELT approach. Its role is limited to transferring the already extracted raw files from the local data directory into MongoDB. No parsing, normalization, deduplication, enrichment, or schema coercion is performed during this phase. All transformation operations are deliberately postponed to later stages, where the data can be cleaned and integrated in a controlled and reproducible manner.

A single generic loader was implemented for all three sources. Instead of developing one loader for each extractor, the loading logic is driven by a source registry. This registry specifies, for each platform, the raw file path, the file format, the natural key used as document identifier, and the destination MongoDB collection. Table 4 reports the resulting loading configuration. This design reduces code duplication and ensures that all sources are loaded through the same operational logic.

For each source, the natural key is used as the MongoDB document identifier. If the same source is loaded multiple times, existing documents are replaced rather than duplicated. The loading strategy also accounts for differences in file size and format. The Google Places file is stored as JSON Lines and is streamed line by line, avoiding full materialization in memory. This is necessary because the file contains 10,808 records and has a size of approximately 249 MB. Conversely, the Tripadvisor and TheFork datasets are stored as JSON arrays and can be loaded in memory due to their smaller size.

During loading, each document is enriched only with minimal technical metadata. In particular, a loading timestamp and the source file path are attached to each record. These metadata fields are separated from the original source attributes and are used exclusively to support traceability and auditing of the loading process.

Records are written to MongoDB in batches using bulk upsert operations. This approach limits memory consumption while reducing the number of database round-

Source	Records	Size	Format	Natural Key	Destination Collection
Google Places	10,808	249 MB	JSON Lines	`place_id`	`restaurants_raw_google`
Tripadvisor	7,539	49 MB	JSON Array	`source_url`	`restaurants_raw_tripadvisor`
TheFork	1,344	11.1 MB	JSON Array	`source_id`	`restaurants_raw_thefork`

Table 4: Configuration adopted by the generic MongoDB loading layer for each data source.

trips. At the end of each loading execution, a structured report is produced, summarizing the number of records read, successfully loaded, updated, and skipped because of missing or invalid natural keys.

4.2 Data Quality Assessment

Before performing any integration, matching or enrichment operation, a preliminary data quality assessment was conducted on the raw datasets collected from Google Places, Tripadvisor, and TheFork. This is a pre-integration assessment: it evaluates the sources as acquired, before the cleaning, geocoding, entity-resolution, and integration stages. The objective was to evaluate the suitability of each source for the subsequent integration phase and to identify potential weaknesses requiring cleaning or enrichment.

The assessment follows standard data-quality dimensions [17], and the metrics were computed according to the following definitions:

- **Completeness**

$$Completeness = 100 \times \frac{\text{Present field values}}{\text{Expected field values}}$$

- **Critical Completeness**

$$C_{\text{crit}} = 100 \times \frac{\text{Present critical fields}}{\text{Expected critical fields}}$$

- **Validity** (format-level check: a present value is valid if it matches its expected source-specific format)

$$V = 100 \times \frac{\text{Valid values}}{\text{Present values checked}}$$

- **Spatial Readiness**

$$S_{\text{spatial}} = 100 \times \frac{\text{Valid-coordinate records}}{\text{Total records}}$$

- **Timeliness**

$$T_{\text{time}} = \max(0,\ 100 \times (1 - r))$$
$$r = \frac{\text{Collection duration}}{\text{Refresh target}}$$

- **Reliability**

$$R = 100 \times \frac{\text{Records with review count } \geq 20}{\text{Total records}}$$

- **Uniqueness**

$$U = 100 - 100 \times \frac{\text{Duplicate records}}{\text{Total records}}$$

- **Score**

$$Score = 0.25C_{\text{crit}} + 0.20V + 0.15S_{\text{spatial}} + 0.15U + 0.10T_{\text{time}} + 0.15R$$

The weights prioritize dimensions that are critical for the project goals. Critical completeness and validity receive the largest weights because missing or invalid identifiers, addresses, ratings, and review counts would directly affect matching and rating comparison. Spatial readiness is also emphasized because geographic evidence is central to the Google-anchored entity-resolution strategy.

Table 5 summarizes the results obtained for the three acquisition sources.

4.2.1 Google Places

The dataset achieved an overall quality score of 93.16%, supported by excellent critical completeness (98.15%), perfect validity (100%) and complete coordinate coverage.

The weakest attributes are website availability (53.23%), price information (77.93%), and phone numbers (83.52%). However, these fields are not essential for entity matching and therefore have a limited impact on integration quality.

A total of 2,937 quality flags were generated (27.17 per 100 records), mainly due to restaurants with a low number of reviews. Nevertheless, the complete availability of geographic coordinates makes Google Places the most suitable source for spatial integration and entity alignment.

4.2.2 Tripadvisor

Tripadvisor contributes a large amount of review-oriented information and contains 7,539 restaurant records. Despite excellent critical completeness (99.78%) and almost perfect validity (99.99%), the source achieved the lowest overall quality score (72.28%).The main limitations are the complete absence of latitude and longitude coordinates in the raw dataset, resulting in a spatial readiness score of 0% and the weakest source refreshability (Timeliness = 43.75%). In addition, review reliability is significantly lower than for the other sources, with 36.12% of restaurants containing fewer than twenty reviews.

The weakest fields are email addresses (46.89%), opening hours (67.60%), and price ranges (67.66%). Further-

Metric	Google	Tripadvisor	TheFork
Records	10808	7539	1344
Quality Score	93.16%	72.28%	97.63%
Completeness	93.79%	84.34%	76.15%
Critical Completeness	98.15%	99.78%	98.88%
Validity	100.00%	99.99%	100.00%
Spatial Readiness	100.00%	0.00%	100.00%
Timeliness	67.23%	43.75%	95.83%
Reliability	79.50%	53.10%	88.84%
Flags / 100 Records	27.17	48.04	17.04

Table 5: Cross-source quality assessment performed before data integration.

more, Tripadvisor exhibits the highest density of quality flags, with 48.04 warnings per 100 records. These results indicate that additional enrichment and geocoding operations are required before reliable integration can be performed.

4.2.3 TheFork

Although TheFork is the smallest dataset (1,344 restaurants), it achieved the highest overall quality score (97.63%). The source exhibits excellent critical completeness (98.88%), perfect validity (100%), complete coordinate coverage (100%), and the highest timeliness score (95.83%).

The primary weaknesses concern contact-related information, as email addresses and phone numbers are not available in the raw dataset, while website coverage is limited to 1.86%. Nevertheless, these attributes are not critical for the integration objectives of this project.

Only 229 quality flags were generated (17 per 100 records), representing the lowest anomaly density among all sources. Consequently, TheFork can already be considered highly suitable for integration and geospatial matching, requiring only limited preprocessing before the subsequent stages of the pipeline. The complete field-level coverage tables and anomaly profile are included in Appendix A.

5 Data Transformation

Following the loading phase, a dedicated transformation layer was developed to convert the raw datasets into clean and analysis-ready collections. The objective of this stage was to improve data consistency, resolve structural heterogeneity across sources and derive additional attributes useful for the subsequent integration process.

5.1 Schema Transformation

The transformation phase follows an ELT paradigm and operates entirely within MongoDB. The raw collections remain immutable and are never modified or overwritten. Instead, each source is processed independently and written into a corresponding clean collection. Figure 2 summarizes this MongoDB-to-MongoDB transformation flow.

5.1.1 Clean Google Dataset Schema

The `google_clean` transformation pipeline produces 10,786 clean records in `restaurants_clean_google`, obtained from 10,808 raw Google records after removing 22 inert junk records. The goal of the cleaning step is to keep only the fields needed for restaurant matching, quality control, and cross-platform rating analysis. The subsequent integration stage further restricts the Google anchor set to records classified as dining venues and operational businesses. Table 6 groups the fields kept in the cleaned Google schema.

Examples of transformations include normalizing `formatted_address` into a cleaner address field, extracting structured address components from Google details, and standardizing city names such as `Milan` to `Milano`. Ratings and review counts are taken from the detailed Google response when available, with fallback to top-level raw values. Several fields are created for data quality control rather than filtering records out immediately. For example, `is_operational` is derived from Google business status, `low_review` marks venues with few reviews, and the `flags` array stores issues such as `non_dining`, `not_operational`, or `city_out_of_area`. Contact fields are standardized to support cross-platform matching. Websites are cleaned by removing URL schemes, leading `www.`, and trailing slashes. Phone numbers are stripped of formatting and converted to the Italian `+39` format when possible. The full raw Google `details` object is not copied into the clean collection; only selected useful fields are extracted, while the raw collection remains available for audit and reproducibility.

5.1.2 Clean Tripadvisor Dataset Schema

The `tripadvisor_clean` transformation pipeline produces 7,539 clean records in `restaurants_clean_tripadvisor` and defines a standardized schema for restaurant matching, quality control, and cross-platform comparison. The cleaning step keeps only analytically useful information from the raw Tripadvisor collection, while replacing raw semi-structured fields with normalized, parsed, or derived attributes. In particular, raw fields such as `price_range`, `cuisine_type`, `working_days_hours`, `review`, `number_photo_uploaded`, and `phone_number` are not copied directly into the clean collection; they

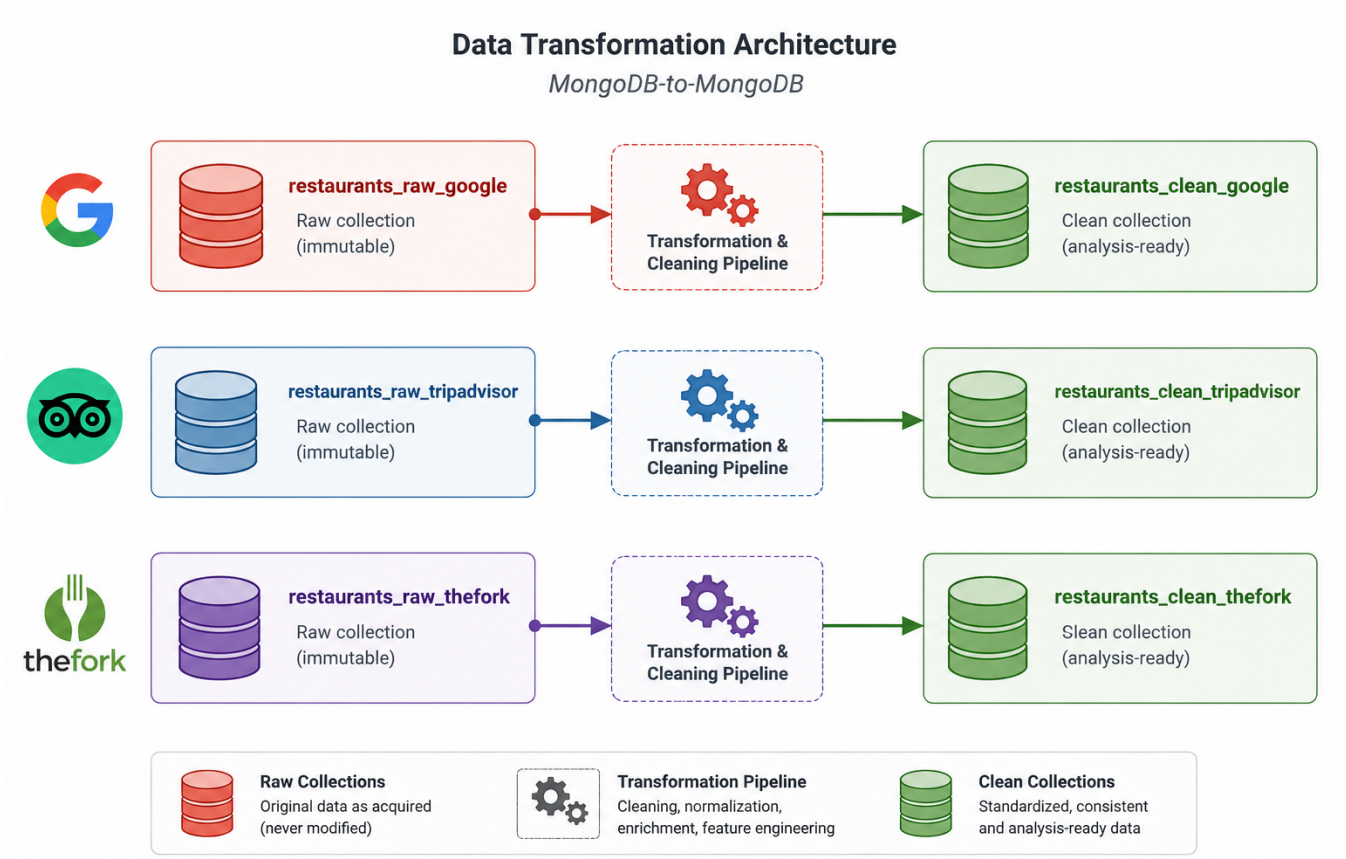


Figure 2: MongoDB-to-MongoDB transformation architecture adopted in the project. Raw collections remain immutable and are transformed into dedicated clean collections through source-specific cleaning, normalization, and feature-engineering procedures.

are converted into structured variables and remain available in the raw collection for audit purposes. Table 7 summarizes the resulting grouped schema.

The transformation pipeline standardizes fields that are originally inconsistent or difficult to use directly. Restaurant names are cleaned by collapsing whitespace, while the Tripadvisor location identifier is extracted from the restaurant URL and used as a stable source-specific key. Address strings are normalized and parsed into street, house number, postal code, and city components, with `has_address` marking address availability. Since Tripadvisor does not provide coordinates in the raw scrape, the cleaned address is geocoded through Nominatim/OpenStreetMap as part of the transformation stage [5]. In the final pipeline, Tripadvisor coordinate coverage reaches 84.0%, and the post-integration assessment reports a median geocoding error of 10.1 m on gold-standard matched rows. Ratings are parsed from Tripadvisor's comma-decimal format, review counts are converted from raw textual values into integers, and coverage indicators are added to distinguish missing information from valid low values. Price ranges are validated and converted into both a source price band and an ordinal tier, while cuisine strings are split, trimmed, and de-duplicated into a list. Opening hours are transformed from raw text into structured daily time intervals, preserving split shifts when present. Review data is reduced to a compact sample containing only selected fields, such as nickname, title, text, date, and contribution count. Contact fields are normalized for matching purposes: websites are stripped of schemes, leading `www.`, and trailing slashes, while Italian phone numbers are converted to a canonical `+39` format when possible. The clean collection therefore avoids copying bulky or redundant raw fields, but preserves the raw Tripadvisor collection as the audit trail for reproducibility.

5.1.3 Clean TheFork Dataset Schema

The `thefork_clean` transformation pipeline produces 1,344 clean records in `restaurants_clean_thefork` and provides a standardized representation of restaurant information for entity matching, data quality assessment, and integration with other restaurant platforms. The cleaning process preserves the most relevant source information while replacing raw or semi-structured attributes with normalized and analytically usable features. In particular, raw fields such as `price_range`, `discount`, `cuisine_type`, `working_days_hours`, and the original nested review structure are transformed into standardized variables, while the raw collection remains available as an audit trail. Table 8 summarizes the corresponding clean TheFork schema.

The transformation pipeline standardizes restaurant names and addresses, extracts a stable TheFork venue identifier from the source key, and canonicalizes loca-

Group	Cleaned information
Identity	Stable identifiers and display name, e.g. `_id`, `place_id`, `name`.
Location	Coordinates and structured address fields, e.g. latitude/longitude, street, house number, postal code, canonical city, province, and country.
Ratings	Google reputation data, including rating, review count, rating availability, and low-review indicator.
Classification	Google venue types transformed into higher-level categories such as restaurant, cafe/bar/bakery, non-dining, or unknown.
Quality flags	Operational and relevance checks, such as non-dining venue, non-operational business, geographic placeholder name, low review count, or out-of-area city.
Contact and price	Normalized website, phone number, price level, and parsed price range; also includes coverage indicators for website and phone availability.
Source richness	Compact evidence about the amount of Google information available, such as photo count and a slimmed sample of recent reviews.
Services and amenities	Optional Google service attributes, such as dine-in, takeout, delivery, reservations, outdoor seating, meal options, drinks, children-friendly service, pets, and restroom.
Metadata	Transformation timestamp and source collection name for traceability.

Table 6: Grouped schema of the cleaned Google restaurant dataset.

Group	Cleaned information
Identity	Stable identifiers and source references, such as `_id`, `source_url`, `ta_location_id`, and `restaurant_name`.
Location	Normalized address information, including `address`, `street`, `house_number`, `postal_code`, `city`, and address coverage flags. Coordinates are included as `latitude` and `longitude` after Nominatim geocoding.
Ratings	Tripadvisor reputation variables, including `rating`, `total_review`, rating and review-count availability indicators, and the `low_review` flag.
Price and cuisine	Parsed price information through `price_band` and `price_tier_level`, together with normalized cuisine categories stored in `cuisines`.
Source richness	Compact indicators of source completeness, including `photo_count`, structured `opening_hours`, `has_hours`, review samples, `sample_size`, and `has_reviews`.
Contacts	Standardized contact fields, including cleaned `website`, normalized `phone`, `email`, and the corresponding coverage indicators.
Quality flags	A consolidated `flags` array storing data quality issues such as `no_rating`, `missing_review_count`, `low_review`, `missing_address`, `missing_coordinates`, `no_reviews`, or `no_hours`.
Metadata	Transformation timestamp and source collection name, stored in `_transformed_at` and `_source_collection`, for reproducibility and traceability.

Table 7: Grouped schema of the cleaned Tripadvisor restaurant dataset.

tion information for consistent matching across platforms. Unlike Tripadvisor, TheFork already provides native geographic coordinates, which are preserved without additional geocoding. Ratings and review counts are validated and converted into numeric variables, while coverage indicators are introduced to distinguish missing information from valid values. Price information is transformed into a numeric average price expressed in euros, and promotional offers are parsed into structured discount percentages whenever possible. Cuisine information is cleaned, de-duplicated, and separated from dietary or religious attributes, which are mapped to standardized tags. Opening hours are converted into structured daily intervals, preserving complex schedules and overnight closures. Review data is reduced to compact review objects containing only the most relevant information, while additional features such as `sample_avg_rating` and `rating_sample_divergent` are generated to support quality assessment. The resulting dataset therefore provides a compact, consistent, and analysis-ready representation of TheFork restaurants while preserving the raw collection for reproducibility and auditing purposes.

5.2 Schema Matching

The objective of the schema matching phase is to identify the semantic correspondences between the clean schemas generated from Google Places, Tripadvisor and TheFork. Unlike entity resolution, which operates at the record level, schema matching focuses on the structural and conceptual alignment of attributes across data sources. The outcome of this phase is a formal set of correspondences and conflict-resolution rules that provide the foundation for the subsequent integration process.

The analysis used the three clean transformation outputs documented above: Google, Tripadvisor, and TheFork. Since each platform models restaurant information according to its own schema and business objectives, direct integration is not possible without first identifying which attributes describe the same real-world concept and which representation conflicts must be resolved.

Schema matching was performed according to two complementary perspectives. The first concerns the semantic relationship between attributes, while the second concerns the representation conflicts that emerge when equivalent concepts are modeled differently. Following

Group	Cleaned information
Identity	Stable identifiers and source references, including `_id`, `source_id`, `tf_id`, `restaurant_url`, and `restaurant_name`.
Location	Native TheFork coordinates and normalized address information, including `latitude`, `longitude`, `address`, `street`, `house_number`, `postal_code`, and canonical city names.
Ratings	TheFork reputation metrics, including `rating`, `review_count`, coverage indicators, and the `low_review` quality flag.
Price and promotions	Structured economic information through `avg_price_eur`, parsed promotional discounts (`discount_pct`), and discount availability indicators.
Cuisine	Normalized cuisine categories stored in `cuisines` together with extracted dietary tags such as vegetarian, vegan, halal, kosher, and gluten-free options.
Opening hours	Structured opening schedules represented as daily opening and closing intervals, together with availability indicators.
Source richness	Additional information describing source completeness, including `photo_count` and review snippets.
Review sample	Slimmed review objects, sample statistics, review coverage indicators, and consistency checks between platform ratings and review-sample ratings.
Quality flags	Data quality indicators such as `no_rating`, `missing_review_count`, `low_review`, `rating_sample_divergent`, and `invalid_cuisine_type`.
Metadata	Transformation timestamp, source collection information, and scraping provenance attributes.

Table 8: Grouped schema of the cleaned TheFork restaurant dataset.

the classical schema integration literature [15], four types of semantic correspondences were considered:

- **Equivalence** ($\equiv$), when two attributes represent the same real-world property;
- **IS-A** ($\sqsubseteq$), when one attribute represents a specialization of a broader concept;
- **Overlap** ($\sqcap$), when two attributes partially intersect but neither subsumes the other;
- **Disjointness** ($\perp$), when two attributes appear similar but actually describe different facts and therefore must never be merged.

The matching process revealed that most restaurant descriptors are semantically equivalent across the three platforms. Restaurant names, addresses, postal codes, cities, ratings, review counts, coordinates, opening hours, and contact information all describe the same underlying concepts, although they are often represented using different structures, naming conventions, or measurement scales. Table 9 gives representative correspondences.

Several representation conflicts were also identified. The most important concerns restaurant ratings. Google Places and Tripadvisor use a 1–5 scale, whereas TheFork adopts a 0–10 scale. Although the three attributes describe the same concept, direct comparison would be misleading without normalization. This represents a scaling conflict rather than a semantic discrepancy. Similarly, price information is represented using heterogeneous formats, including categorical price levels, symbolic price bands, and explicit average prices expressed in euros. Cuisine descriptors are an additional semantic conflict, since the three platforms use different and even cross-language vocabularies; this overlap is resolved later through an explicit canonical cuisine taxonomy applied during dataset integration (Section 7.4).

Additional conflicts arise from structural differences. Address information may be represented either as a single normalized string or as a set of decomposed components. Review data exhibit even greater heterogeneity, since each platform provides different review structures, metadata, and levels of detail. Consequently, review objects are preserved together with their source provenance rather than being merged directly. Table 10 summarizes the main conflict types and their resolution strategies.

A particularly important outcome of the analysis concerns source-specific identifiers. Google Places, Tripadvisor, and TheFork provide stable identifiers within their own platforms, namely `place_id`, `ta_location_id`, and `tf_id`. Although these identifiers uniquely identify restaurants within each source, they belong to independent namespaces and therefore cannot be used as integration keys. Entity resolution must consequently rely on descriptive attributes such as restaurant names, addresses, coordinates, and contact information.

The correspondences are not only descriptive: they determine the actual integrated schema. Google identity and coordinates are used as the canonical restaurant backbone; TheFork's native 0–10 rating is normalized into a comparable `thefork_rating_5`; Google, Tripadvisor, and TheFork ratings are intentionally preserved as separate attributes because rating disagreement is the object of analysis; price signals from Google price levels, Tripadvisor price bands, and TheFork average prices are mapped to a common price representation; and review objects are not fused into a single text field because their structures, sampling rules, and provenance differ across platforms. The schema matching phase therefore establishes the semantic foundation required for integration. By explicitly defining attribute correspondences, identifying representation conflicts, and documenting source-specific characteristics, it becomes possible to generate consistent mapping rules and construct a unified restaurant schema without ambiguity or loss of information.

Integrated Concept	Relation	Source Attributes
Restaurant name	$\equiv$ Equivalence	`name`, `restaurant_name`
Address information	$\equiv$ Equivalence	`address`, `street`, `postal_code`, `city`
Geographic coordinates	$\equiv$ Equivalence	`latitude`, `longitude`
Restaurant rating	$\equiv$ Equivalence	`rating`
Review count	$\equiv$ Equivalence	`review_count`, `total_review`
Dietary information	$\sqsubseteq$ IS-A	`dietary_options`, `serves_vegetarian_food`
Cuisine descriptors	$\sqcap$ Overlap	`cuisines`, `primary_type`
Platform identifiers	$\perp$ Disjointness	`place_id`, `ta_location_id`, `tf_id`

Table 9: Examples of semantic correspondences identified during schema matching.

Conflict Type	Example	Resolution Strategy
Naming conflict	`name` vs `restaurant_name`	Canonical attribute naming
Scaling conflict	Ratings 1–5 vs 0–10	Rating normalization
Structural conflict	Address strings vs structured components	Schema harmonization
Semantic conflict	Regional vs national cuisine labels	Vocabulary mapping
Coverage conflict	Missing coordinates in Tripadvisor	Geocoding and enrichment

Table 10: Main conflicts identified during schema matching.

6 Entity Resolution

Following schema matching, the next challenge consists of identifying records that refer to the same real-world restaurant across different platforms. Since Google Places, Tripadvisor and TheFork use independent identifiers and maintain separate data repositories, no direct integration key exists. Consequently, entity resolution was implemented as a dedicated record-linkage process [16] designed to identify matching restaurants while minimizing both false positives and false negatives.

The entity resolution workflow operates on the clean collections generated during the transformation phase. Google Places was selected as the anchor source due to its superior geographic coverage, high data quality score, and complete coordinate availability. Candidate matches are therefore generated only between Google and Tripadvisor records and between Google and TheFork records. Direct Tripadvisor–TheFork matching is intentionally avoided because both platforms exhibit lower coverage and are expected to be linked indirectly through Google. Figure 3 summarizes this Google-anchored workflow.

6.1 Blocking Strategy

A naïve comparison between all records would require evaluating millions of candidate pairs, making the process computationally inefficient. To reduce the search space, a blocking phase was introduced before similarity computation.

The primary blocking strategy is based on geographic proximity. For records containing valid coordinates, candidate pairs are generated only when the distance between the Google restaurant and the corresponding Tripadvisor or TheFork restaurant is below 150 meters. Geographic distance is computed using the Haversine formula, which provides an accurate estimate of distance between two latitude–longitude pairs.

A secondary blocking strategy is applied only to Tripadvisor records that lack geographic coordinates. In this case, candidate pairs are generated when restaurants share the same postal code and at least one common name token of four or more characters. This additional constraint prevents excessively large postal-code blocks from generating an impractical number of comparisons while preserving a high probability of capturing true matches.

Records lacking both coordinates and postal-code information cannot be reliably compared using any available blocking key. These records are therefore explicitly marked as `UNBLOCKABLE` and preserved for auditing purposes.

6.2 Similarity Computation

For each candidate pair generated during the blocking phase, a set of similarity features is computed. The most important signal is restaurant-name similarity, obtained through token-based fuzzy matching implemented with RapidFuzz after additional normalization steps such as abbreviation expansion, punctuation removal, and whitespace standardization [11].

Additional comparison features include street-name similarity, postal-code agreement, geographic distance, phone-number equality, and website equality when such information is available. Phone numbers and websites are particularly valuable for Tripadvisor matching because they provide strong evidence of identity when identical values are observed across sources.

Cuisine information is also evaluated through set-based similarity measures. However, due to the vocabulary heterogeneity discussed in the schema matching phase, cuisine similarity is used only as a diagnostic feature and does not directly contribute to the final matching score. Table 11 summarizes the similarity features retained for candidate evaluation.

6.3 Candidate Classification

Similarity features are combined into a weighted composite score representing the likelihood that two records correspond to the same restaurant. Different weight-

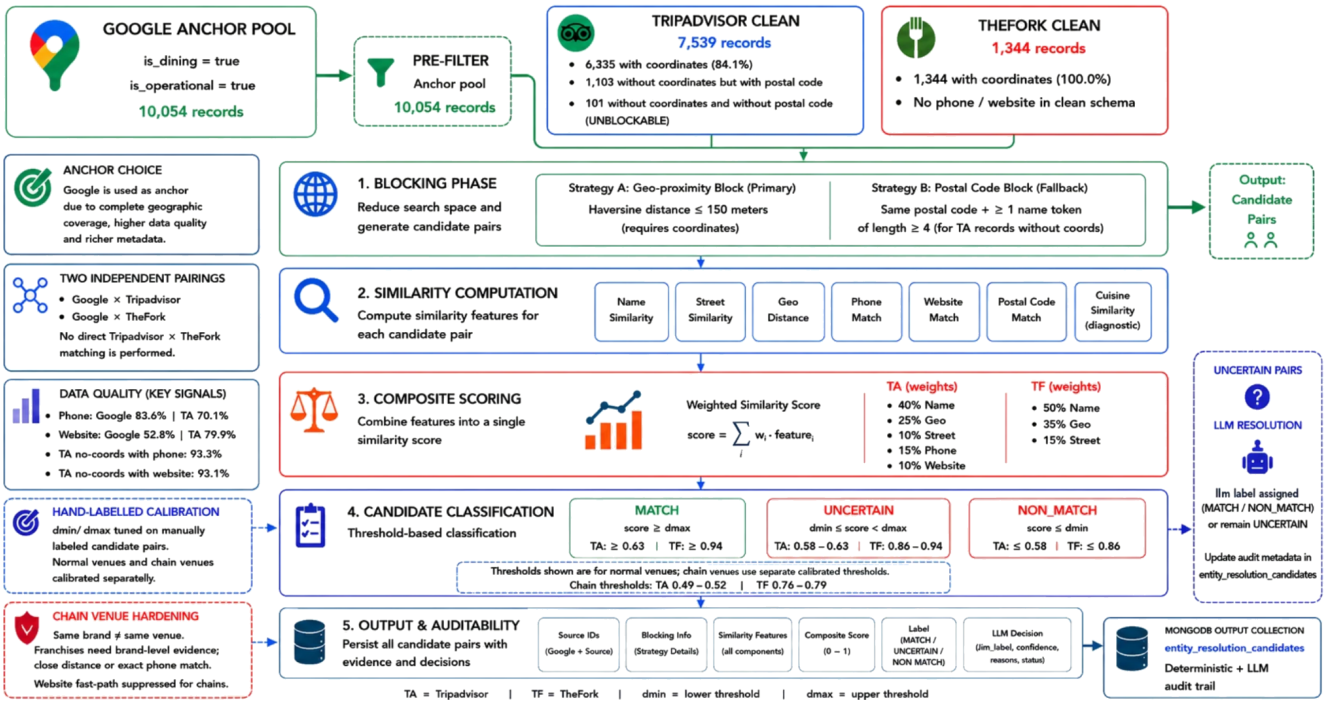


Figure 3: Entity resolution workflow adopted for restaurant linkage across Google Places, Tripadvisor and TheFork. Google acts as the anchor dataset, candidate pairs are generated through blocking strategies, evaluated using similarity features, and classified into MATCH, NON_MATCH, or UNCERTAIN categories. Uncertain pairs are forwarded to a subsequent LLM-based resolution stage.

Feature	Type	Purpose
Name similarity	Fuzzy matching	Primary identity signal
Street similarity	Fuzzy matching	Address consistency check
Geographic distance	Spatial similarity	Distinguishes nearby venues
Phone match	Exact match	Strong matching evidence
Website match	Exact match	Strong matching evidence
Postal-code match	Exact match	Geographic consistency check
Cuisine similarity	Diagnostic only	Analytical support feature

Table 11: Similarity features used during entity resolution.

ing schemes are adopted for Tripadvisor and TheFork because TheFork does not provide phone numbers or restaurant websites in the clean schema. For Tripadvisor the score weights name similarity at 0.40, geographic proximity at 0.25, phone match at 0.15, street similarity at 0.10, and website match at 0.10. For TheFork, where contact signals are unavailable, the weight is redistributed over the remaining features, namely name similarity at 0.50, geographic proximity at 0.35, and street similarity at 0.15. In both cases name similarity is the dominant identity signal, complemented by geographic proximity.

The resulting score is compared against two calibrated thresholds. Candidate pairs with scores above the upper threshold are automatically classified as `MATCH`, while pairs below the lower threshold are classified as `NON_MATCH`. Records falling between the two thresholds are assigned to an intermediate `UNCERTAIN` category.

Thresholds were calibrated from manually labelled candidate samples. Normal venues and chain venues were treated separately because repeated brands, such as fast-food or casual dining chains, require stricter branch-level evidence than independent restaurants. The calibration files are kept as reproducible gold-standard artifacts and are not used as hidden runtime inputs by the entity-resolution command.

This three-way classification strategy was intentionally adopted to reduce the risk of incorrect automatic decisions. Rather than forcing all pairs into a binary outcome, ambiguous cases are isolated and delegated to a subsequent resolution stage.

An additional fast-path mechanism is implemented for Tripadvisor records without coordinates. When phone numbers or websites are identical across sources, the candidate pair can be classified as a `MATCH` without requiring further similarity computation, subject to the conservative chain-handling rules described below. This optimization significantly improves both efficiency and precision because identical contact information provides strong evidence that two records refer to the same establishment.

6.3.1 Chain and Franchise Handling

Restaurant chains and franchises require a dedicated matching strategy because the usual identity signals become unreliable when the same brand operates several branches in Milan. Two different branches of the same franchise, such as two *KFC*, *Poke House*, or *McDonald's* venues, share almost identical names and often the

Candidate type	Lower threshold	Upper threshold
Tripadvisor, non-chain venues	0.58	0.63
TheFork, non-chain venues	0.86	0.94
Tripadvisor, chain venues	0.49	0.52
TheFork, chain venues	0.76	0.79

Table 12: Calibrated entity-resolution thresholds used for automatic candidate classification.

same corporate website, yet they are distinct real-world restaurants. The guiding principle is therefore that *the same brand does not imply the same venue*: for chain candidates, name and website similarity are no longer sufficient evidence of a match, and branch-level evidence is required instead.

Chain candidates are first identified through brand detection. A curated list of chain brands observed among the repeated Google anchor names in the Milan dataset (for example *KFC*, *Poke House*, *McDonald's*, *Five Guys*, *Spontini*, *Starbucks*, and *Roadhouse*) is matched against the normalized restaurant name. Generic repeated names that do not identify a specific brand, such as *bar* or *pizzeria*, are deliberately excluded. A candidate pair is flagged as a chain when either the Google record or the source record carries a recognized brand.

Chain candidates are then subject to two hardening rules on top of the stricter calibrated thresholds already reported in Table 12. First, the website fast-path is suppressed for chains: because all branches of a franchise typically expose the same corporate website, website equality is not accepted as proof of identity, while the more branch-specific phone fast-path remains valid. Second, a chain pair that would otherwise be classified as `MATCH` is downgraded to `UNCERTAIN` when no geographic evidence is available, or when the two venues are farther apart than a small automatic-match radius (75 m by default). This ensures that automatic chain matches are only accepted when the two records are geographically compatible with being the same branch. Every hardening decision is recorded in a `chain_hardening` field, so the more conservative treatment of franchises remains fully auditable.

6.4 Output and Auditability

The output of the entity resolution process is stored in the MongoDB collection `entity_resolution_candidates`. Each candidate pair preserves the identifiers of both source records, the blocking strategy used to generate the candidate, the individual similarity components, the composite score, and the final classification label.

Maintaining both positive and negative decisions provides a complete audit trail of the matching process. Furthermore, uncertain candidates are retained together with all supporting evidence, enabling a subsequent large-language-model resolution step without requiring candidate generation to be repeated.

The resulting collection therefore acts as the bridge between schema-level integration and the construction of the final integrated restaurant dataset, providing a transparent and reproducible record-linkage layer for the entire project.

6.5 Threshold Calibration

The decision thresholds were calibrated against a manually labelled gold standard. The gold standard was constructed by sampling candidate pairs generated during blocking and exporting each of them together with its full matching evidence, including the two restaurant names, addresses, coordinates, the individual similarity components, and the automatic label produced by the pipeline. Each sampled pair was then reviewed by hand and assigned a human label of `MATCH` or `NON_MATCH`, giving a ground truth against which the automatic decisions can be compared.

Two disjoint gold subsets were used, following a calibrate-then-verify protocol. The calibration subset contains 600 labelled candidate rows, split between normal venues and chain venues, and is used exclusively to choose the lower and upper decision thresholds ($d_{\min}$ and $d_{\max}$) reported in Table 12. Because the thresholds are fitted on this subset, its metrics alone would be an optimistic estimate of quality. For this reason a second, additional set of candidate pairs was hand-labelled purely for verification: a separate out-of-sample (holdout) subset of 288 rows, obtained after de-duplication and after excluding any candidate already present in the calibration files. This holdout is never used to choose $d_{\min}$ or $d_{\max}$; it only verifies the already-calibrated thresholds, providing an unbiased estimate of how the classifier behaves on unseen pairs. The classifier error on this holdout, together with one-to-one link survival and Tripadvisor geocoding accuracy, is reported as part of the post-integration quality assessment in Section 7.5.

6.6 LLM based labelling on Uncertain Pairs

While the deterministic entity resolution process is able to automatically classify the majority of candidate pairs as either `MATCH` or `NON_MATCH`, a subset of cases remains inherently ambiguous. These candidate pairs fall within the intermediate decision region defined by the three-way classification policy and are labelled as `UNCERTAIN`. Rather than forcing a potentially unreliable automatic decision, the implemented pipeline provides an optional second adjudication layer based on a Large Language Model (LLM), applied exclusively to these unresolved cases.

The LLM component does not replace the deterministic record linkage process and does not operate on the full space of possible source combinations. Instead, it acts

as a controlled semantic reasoning layer on top of the candidate pairs already generated through blocking and similarity scoring. Consequently, the computational cost and decision scope remain limited to a relatively small set of ambiguous records. Only documents in the collection `entity_resolution_candidates` with `label = UNCERTAIN` and no previous LLM decision are considered for processing.

Candidate pairs are grouped according to the source record identifier, so that the model evaluates a single Tripadvisor or TheFork restaurant together with its competing Google candidates rather than examining pairs independently. For each group, Google candidates are ranked according to their deterministic score, geographic proximity, name similarity, and street similarity. The prompt then presents the source restaurant together with the highest-ranked candidates and all relevant matching evidence, including names, addresses, coordinates, contact information, ratings, cuisine descriptors, and similarity components.

The model is instructed to select one of three possible outcomes: `MATCH`, when exactly one candidate can be confidently identified as the same physical restaurant; `NON_MATCH`, when none of the candidates appear plausible; or `UNCERTAIN`, when the available evidence is insufficient or multiple candidates remain equally plausible. To ensure reproducibility and machine-readable outputs, responses are constrained to a predefined JSON schema containing the final decision, the selected candidate identifier, a confidence value, a textual explanation, and a set of predefined risk flags describing potential sources of ambiguity.

A key design principle of the implemented approach is that LLM decisions are never accepted blindly. Every response is subjected to a deterministic post-validation policy before being incorporated into the integration workflow. Candidate matches may be downgraded from `MATCH` to `UNCERTAIN` when the confidence value falls below a configurable threshold, when the selected candidate does not belong to the candidate set presented in the prompt, when severe risk flags are detected, or when geographic evidence is inconsistent with the absence of supporting contact information. This validation layer ensures that the generative component remains subordinate to conservative and transparent decision rules.

The resulting decision is stored as an additional layer of metadata rather than replacing the original deterministic classification. Fields such as `llm_label`, `llm_confidence`, `llm_reason`, `llm_risk_flags`, and `llm_prompt_version` are persisted together with the original similarity evidence, enabling full traceability of the adjudication process. The downstream unification command consumes `llm_label` when it is available; otherwise, it falls back to the deterministic label. Consequently, deterministic matches remain valid, uncertain pairs promoted by the LLM can contribute to the final integrated dataset and unresolved ambiguities remain excluded from automatic integration.

7 Schema Integration and Analytical Storage

After schema matching and entity resolution, the project materializes a unified restaurant dataset. This stage is the point at which the three platform-specific clean collections become a single integrated database suitable for quality assessment and analytical querying.

7.1 Construction of the Integrated Dataset

The integration process is implemented as a MongoDB-to-MongoDB transformation. It reads the three clean collections and the entity-resolution candidates:

- `restaurants_clean_google`;
- `restaurants_clean_tripadvisor`;
- `restaurants_clean_thefork`;
- `entity_resolution_candidates`.

The output consists of two collections:

- `entity_resolution_links`, containing the selected one-to-one links between Google anchors and matched source records;
- `restaurants_integrated`, containing the final integrated restaurant entities.

The effective candidate label, stored as `effective_label`, is computed as:

$$L_{\text{eff}} = \begin{cases} \texttt{llm_label}, & \text{if an LLM decision is available} \\ \texttt{label}, & \text{otherwise} \end{cases}$$

Only candidates whose effective label is `MATCH` are eligible for link selection. Tripadvisor and TheFork links are selected independently, so an integrated restaurant may contain only Google evidence, Google plus Tripadvisor, Google plus TheFork, or all three platforms.

7.2 Final Integrated Schema

The integrated collection is Google-seeded: each document represents one dining and operational Google restaurant. The document identifier is deterministic:

$$\texttt{integrated_restaurant_id} = \texttt{"google:"} + \texttt{place_id}$$

Google is used as the canonical identity and geographic backbone because it has the highest spatial coverage and complete coordinates. Non-Google sources are attached as evidence blocks only when selected entity-resolution links exist. Table 13 summarizes the main field groups of the integrated collection.

The schema deliberately preserves source-specific ratings rather than replacing them with a single resolved

Field group	Main attributes
Canonical identity and geography	integrated_restaurant_id, canonical_name, canonical_address, canonical_city, latitude, longitude.
Platform membership	has_google, has_tripadvisor, has_thefork, platform_count.
Comparable ratings	google_rating_5, tripadvisor_rating_5, thefork_rating_5, rating_avg_5, rating_range_5.
Review counts	google_review_count, tripadvisor_review_count, thefork_review_count.
Contacts and price	website, phones, price_level, price_evidence.
Cuisine and taxonomy	cuisine_primary, cuisine_tags.
Audit and provenance	integration_flags, sources.google, sources.tripadvisor, sources.thefork.

Table 13: Structure of the final `restaurants_integrated` collection.

value. This is a central design decision: the research questions require measuring disagreement across platforms, therefore the differences between Google, Tripadvisor, and TheFork must remain visible.

7.3 Conflict Handling and Provenance

Conflicts are resolved according to the role of each attribute. Canonical identity and coordinates use a metadata-based strategy: Google is trusted as the anchor source. Comparative values, such as ratings and review counts, are not resolved away; they are kept side by side. Derived fields such as `rating_avg_5` and `rating_range_5` summarize the comparison without hiding the source values.

Contacts and price are handled differently. Phone values are preserved as a deduplicated set because a restaurant may legitimately have multiple numbers. Website conflicts keep the raw evidence and prefer Google only when conflicting values cannot be reconciled. Price is a genuine value conflict, because the three platforms may disagree on how expensive a restaurant is. Each source price is first mapped to a shared ordinal tier expressed with human-readable labels (`INEXPENSIVE`, `MODERATE`, `EXPENSIVE`, `VERY_EXPENSIVE`). The top-level price level is then resolved by majority-vote consensus across the available sources rather than by a fixed source priority: the tier supported by the largest number of platforms wins. When two tiers are equally supported, the conflict is not forced; the resulting value is a sorted list of the tied labels, which deliberately preserves the signal that the restaurant straddles price tiers across platforms. The original source-specific price values remain available under the source blocks.

This design maintains reproducibility: every integrated top-level value can be traced back to the original platform evidence stored under `sources.google`, `sources.tripadvisor`, and `sources.thefork`.

7.4 Cuisine Reconciliation

Cuisine was the most heterogeneous descriptor to integrate, and it is resolved explicitly during the construction of the integrated dataset. The three sources express cuisine in three unreconciled vocabularies: Tripadvisor uses Italian feminine adjectives (*Italiana*, *Asiatica*), TheFork uses masculine adjectives (*Italiano*, *Asiatico*) together with many regional Italian labels (*Lombardo*, *Toscano*, *Napoletano*), while Google exposes an English controlled vocabulary (`italian_restaurant`, `pizza_restaurant`) that covers almost every venue but is dominated by non-food types. A first version of the integration kept only the first cuisine label of each venue, which split a single cuisine across several rows and ignored Google's near-complete coverage. The reconciliation step was introduced to remove this bias.

The reconciliation is implemented as an explicit, hand-built canonical taxonomy rather than automatic stemming. Stemming cannot translate Google's English labels and would incorrectly merge unrelated terms such as *Pesce* (fish) and *Pizza*, whereas an explicit map also documents each judgement call and keeps the step auditable. The map is derived from the complete exploded inventory of all observed labels across the three platforms (51 Tripadvisor labels, 57 TheFork labels, and roughly 130 Google food types) and assigns every raw label to a small set of canonical buckets. Regional Italian labels collapse into *Italian*, and cross-language variants such as *Italiana*, *Italiano*, and `italian_restaurant` converge onto the same bucket.

Because most venues carry several cuisines, the reconciliation is multi-label. The union of canonical buckets observed across the three platforms is kept as `cuisine_tags`, while a single `cuisine_primary` is derived for analyses that require one point per venue. The primary bucket is chosen through a specificity-ranked consensus: the most specific bucket wins, so *Pizza* takes precedence over the umbrella *Italian* and *Chinese* over the umbrella *Asian*. Ties are broken by source precedence, preferring Tripadvisor over TheFork and TheFork over Google. An agreement indicator records whether the platforms concur on the venue's cuisine. This reconciliation is the data-management step that later allows the research-question analysis to group restaurants on a single trustworthy cuisine bucket instead of raw, gendered, or language-specific labels.

7.5 Post-Integration Quality Assessment

The quality of the integration was evaluated against the same manually labelled gold standard described in the entity-resolution chapter, comparing the pipeline's automatic decisions with the human `MATCH`/`NON_MATCH` labels. The assessment measures not only the entity-resolution classifier, but also the survival of one-to-one selected links through the integration step and the spatial enrichment of Tripadvisor records, so that the final integrated dataset is judged end to end rather than only at the matching

stage. Table 14 reports the resulting post-integration metrics.

The results show a conservative integration policy. Precision is high, meaning that automatic matches are rarely wrong. Strict recall is lower because many true matches are intentionally left as `UNCERTAIN` rather than forced into unsafe automatic links. This behaviour is desirable in a data integration setting: false positives would contaminate the final dataset more severely than unresolved candidates.

The survival rate quantifies the effect of one-to-one selection. Some human-confirmed matches are not present in the final integrated source blocks because conservative link selection removes ambiguous alternatives. The geocoding assessment confirms that Tripadvisor spatial enrichment is reliable enough for blocking and spatial analysis: the median error is 10.1 m and all evaluated gold rows are within 100 m. The supporting gold-sample counts, metric definitions, survival tables, and geocoding-error diagnostics are included in Appendix B.

7.6 ClickHouse Analytical Layer

MongoDB is well suited for raw and integrated document storage, but the final research questions require repeated aggregations, pairwise rating comparisons, ranking, grouping by geographic area, and correlation-style summaries. For this reason, the project adds ClickHouse as a column-oriented analytical DBMS [7].

The ClickHouse loader reads the cleaned and integrated MongoDB collections and creates four flat analytical tables:

- `restaurants_integrated`;
- `restaurants_clean_google`;
- `restaurants_clean_tripadvisor`;
- `restaurants_clean_thefork`.

The load process is reproducible and idempotent: each run creates the destination tables if needed, truncates them, and reloads the current MongoDB state. The integrated table keeps the analytical columns needed for the research questions, including platform ratings, review counts, platform membership flags, normalized price tier, the canonical cuisine fields (`cuisine_primary` and `cuisine_tags`) produced by the reconciliation step, photo counts, and geographic coordinates. Heavy nested evidence remains in MongoDB, while ClickHouse stores a flat projection optimized for SQL analysis.

The design therefore separates responsibilities:

- MongoDB stores raw, clean, integrated, and auditable document data;
- ClickHouse stores flat analytical tables for fast, reproducible queries;
- the notebook reads from ClickHouse without modifying either database.

This two-layer architecture satisfies the project requirement of modelling and storing integrated data in DBMSs while also providing an efficient surface for the final research-question analysis.

8 Research Questions Analysis

The final integrated dataset contains 10,054 restaurants. In particular, 5,967 restaurants are available only on Google, 3,179 on Google and Tripadvisor, 163 on Google and TheFork, and 745 on all three platforms. This difference in coverage must be considered when comparing ratings, because the platforms do not describe exactly the same subset of restaurants. Building on this integrated database, and on the ClickHouse analytical layer, this chapter answers the three research questions introduced earlier. Each question is expressed as a SQL query over the flat analytical tables and summarized through a dedicated visualization.

8.1 RQ1: Platform Rating Bias

For restaurants with at least two platform ratings, the ratings are generally close. The average difference between the highest and lowest rating is 0.518 stars; 65.1% of restaurants differ by at most 0.5 stars and 89.6% differ by at most 1 star. The strongest agreement is between Google and TheFork, with an average absolute difference of 0.246 stars. Comparisons involving Tripadvisor show larger differences: 0.494 stars for Google–Tripadvisor and 0.482 stars for Tripadvisor–TheFork. The most relevant result is therefore not that ratings are unrelated, but that Tripadvisor tends to be the platform that creates the largest cross-platform gaps.

Figure 4 focuses on this platform effect. The plot uses a leave-one-out (LOO) comparison: each platform is compared with the average rating given by the other available platforms for the same restaurants. A positive value means that the platform tends to rate restaurants higher than the others; a negative value means that it tends to rate them lower. Tripadvisor is the only platform clearly below zero, with a LOO value of -0.362 stars. Google and TheFork are above zero, with values of 0.308 and 0.180 stars respectively. This confirms that Tripadvisor is systematically more severe in the integrated dataset, while Google and TheFork are more positive.

8.2 RQ2: Rating Level and Consistency by Cuisine

Restaurant category affects both rating level and rating consistency. Figure 5 compares cuisine types using the canonical taxonomy produced after the cuisine reconciliation step. Pizza, fast-food and American-style venues appear among the categories with lower average ratings and larger cross-platform disagreement. By contrast, seafood, Japanese and African cuisines are associated with higher and more consistent ratings. Rating level and cross-platform agreement therefore tend to rise together:

Metric	Value
Out-of-sample evaluation rows	288
In-calibration labelled rows	600
MATCH precision	96.8%
Strict MATCH recall	71.7%
MATCH-or-UNCERTAIN kept recall	100.0%
Uncertain rate	33.0%
Selected links from human-confirmed matches	88 / 127
Link survival rate	69.3%
Integration survival rate	69.3%
Median Tripadvisor geocoding error	10.1 m
Tripadvisor coordinates within 50 m	91.3%
Tripadvisor coordinate coverage	84.0%

Table 14: Post-integration assessment of entity resolution, link selection, and Tripadvisor geocoding.



Figure 4: Platform bias measured with a leave-one-out comparison. Values above zero indicate ratings higher than the other platforms; values below zero indicate lower ratings.

mainstream, high-volume cuisines are both lower-rated and more contested, whereas specialist cuisines are higher-rated and more aligned across platforms. The canonical buckets make the pattern clearer than the older raw-label view and remove the need to interpret separate labels such as *Italiana* and *Italiano* as distinct cuisine groups.

8.3 RQ3: Location and Data Completeness

Location strongly affects data completeness. Completeness is quantified with a composite score, defined as the mean presence rate over the platform-provided facets that genuinely vary across the city: whether the restaurant has a website, whether it carries a cuisine tag, and whether it is present on Tripadvisor and on TheFork. The candidate signals were audited beforehand, and facets that are already saturated everywhere, such as Google reviews, photos, and phone numbers, were excluded because they do not discriminate between central and peripheral venues. The resulting score is not distributed uniformly across Milan but depends strongly on where the restaurant is located. Restaurants closer to the centre of the city are described more completely. In the central area, the average number of platforms per restaurant is 1.67, compared with 1.53 in the periphery. Tripadvisor covers 53.1% of central restaurants and 43.3% of peripheral restaurants; TheFork covers 13.9% of central restaurants and 9.3% of peripheral restaurants. Central restaurants also have more Google reviews, with a median of 269 compared with 161 in peripheral areas. Data completeness is therefore spatially structured: central districts exhibit substantially richer metadata than peripheral areas.

Figure 6 makes this spatial completeness gradient visible by mapping the composite completeness score across Milan. Each cell is coloured by the average completeness

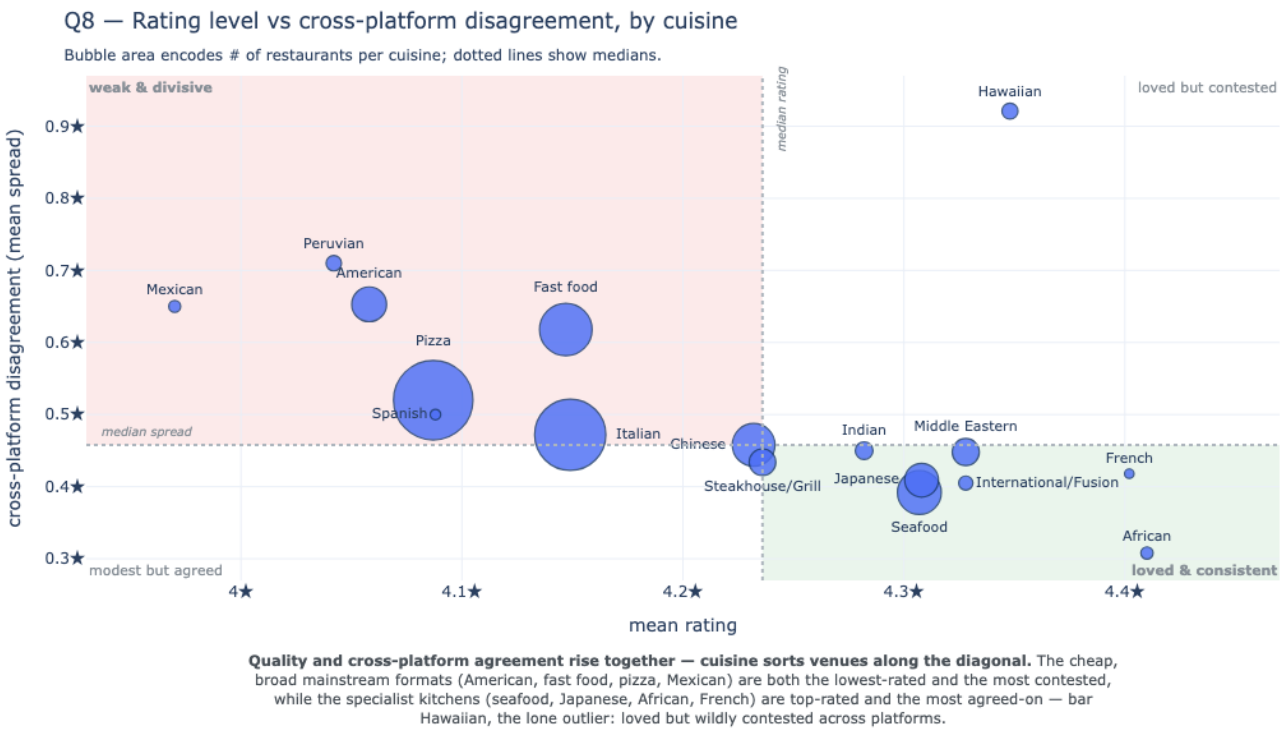


Figure 5: Cuisine categories differ both in average rating and in rating disagreement across platforms.

of the venues it contains, from about 25% (dark blue) to about 65% (light yellow). The historic core is data-rich, with completeness above roughly 47%, and completeness peaks in the Navigli and Isola neighbourhoods before fading smoothly to around 30% in the outer districts. The map therefore confirms that the richest metadata is concentrated in the central part of the city and decreases towards the periphery.

8.4 Further Analyses

The three research questions above are the focus of this report. The integrated dataset is nonetheless designed to support a broader set of analytical questions, which are explored in depth as part of the data-visualization module of the course, including:

- How consistent are restaurant ratings across platforms overall?
- Which restaurants exhibit the highest disagreement across platforms?
- Is rating inconsistency linked to data-quality issues?
- Can sparse, low-review data inflate perceived quality?
- Does rating inconsistency rise for less popular venues?
- Do pricier restaurants tend to rate higher and more consistently?
- Are multi-platform venues rated differently?
- Does photo richness track popularity or rating?

Each of these questions is backed by dedicated SQL over the ClickHouse analytical tables and a corresponding notebook in the project repository, confirming that the integrated schema is expressive enough to sustain them without any further data collection.

8.5 Summary

Overall, the integrated database shows that online restaurant ratings in Milan are usually similar across platforms, but not identical. Tripadvisor is systematically more severe than Google and TheFork (RQ1); cuisine influences both rating level and cross-platform agreement, with mainstream categories such as pizza and fast food being lower-rated and more contested than specialist cuisines (RQ2); and data completeness is spatially structured, with central districts described far more richly than peripheral areas (RQ3). The integration of the three sources therefore makes it possible to identify not only average restaurant reputation, but also the cases where that reputation depends strongly on the platform, the cuisine, and the location.

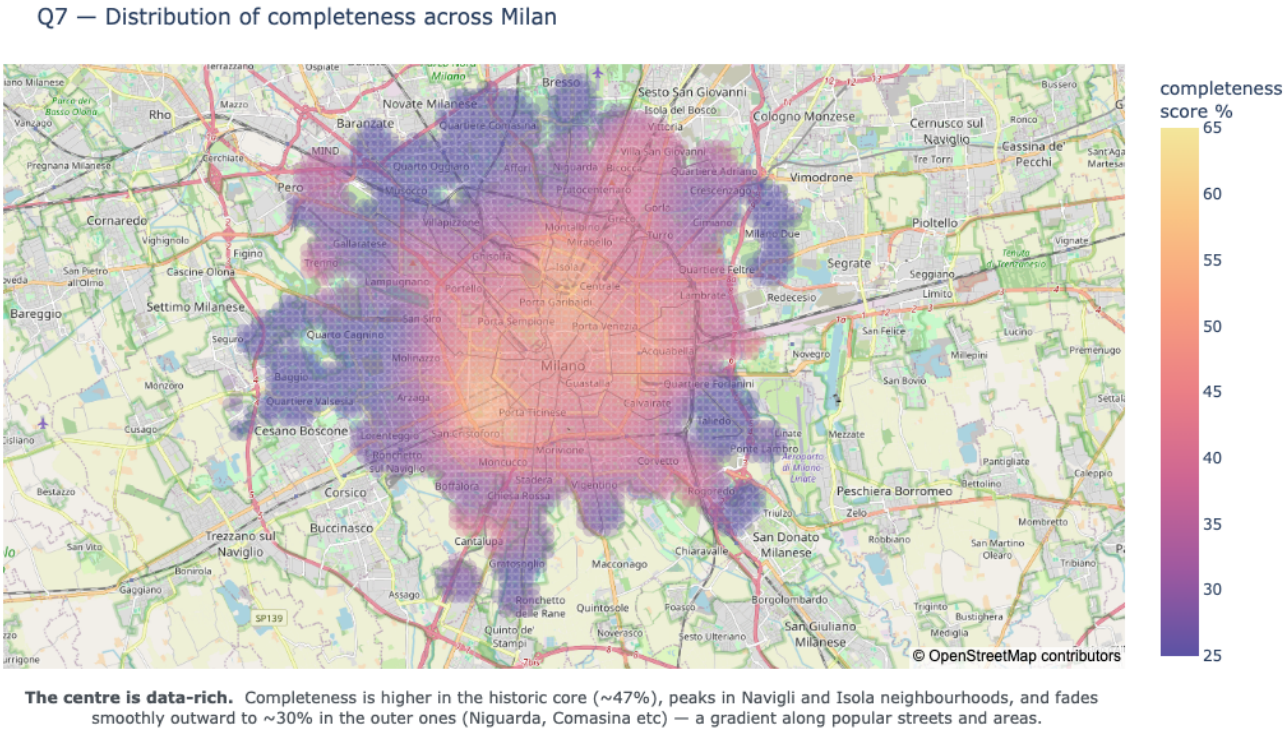


Figure 6: Composite data-completeness score across Milan. Warmer colours indicate higher completeness; the score peaks in the historic centre and the Navigli/Isola belt and fades towards the periphery.

9 Conclusions

This project built and evaluated a complete data-management pipeline for comparing restaurant ratings across Google Maps, Tripadvisor, and TheFork in the Milan area. Starting from three heterogeneous sources with independent identifiers, formats, and levels of quality, the work produced a single integrated database suitable for consistency analysis and analytical querying.

9.1 Summary of the Work

The pipeline follows an ELT design. Raw data are acquired through the Google Places API and two dedicated scrapers, loaded into MongoDB as an immutable system of record, and then cleaned into per-source collections. Schema matching aligns the three vocabularies and classifies their representation conflicts, after which a Google-anchored entity-resolution stage links records through blocking, similarity scoring, and calibrated thresholds, with dedicated hardening for chain and franchise venues and an optional large-language-model adjudication of uncertain pairs. The selected links are materialized into a Google-seeded integrated collection that keeps each platform's ratings side by side, reconciles cuisine into a canonical taxonomy, and resolves price by majority-vote consensus. Finally, the cleaned and integrated collections are projected into a flat ClickHouse layer that supports the research-question queries.

Quality was measured at two distinct points. Before integration, a data-quality assessment profiled each raw source along standard data-quality dimensions. After integration, the assessment targeted the integration process itself, evaluating the entity-resolution classifier, one-to-one link survival, and Tripadvisor geocoding against a manually labelled gold standard.

9.2 Main Findings

The integrated dataset contains 10,054 restaurants. Ratings are generally consistent across platforms, but the agreement is not perfect and its structure is informative. Tripadvisor is systematically more severe than Google and TheFork, which are closely aligned with each other (RQ1). Rating level and cross-platform agreement vary by cuisine, with mainstream categories such as pizza and fast food being lower-rated and more contested than specialist cuisines (RQ2). Data completeness is spatially structured, with the richest metadata concentrated in the historic centre and the Navigli and Isola belt and decreasing towards the periphery (RQ3).

9.3 Limitations

Several limitations follow from the design. The dataset is Google-seeded, so a restaurant absent from Google cannot enter the integrated collection, and TheFork's small footprint limits three-platform coverage. Tripadvisor coordinates are obtained through geocoding rather than measured, which introduces a small spatial error. Entity resolution is deliberately conservative: many true matches are left as uncertain rather than forced, and one-to-one link selection reduces the survival of some human-confirmed matches. Finally, the data represent a single point in time and do not capture how ratings evolve.

9.4 Future Work

The integrated dataset opens several extensions. A first group of additional research questions — the effect of review volume on rating stability, the role of price level, the impact of multi-platform presence on visibility, and the relationship between photo activity and engagement — is being carried forward as part of the data-visualization module of the course. Beyond these, the most natural extension is to exploit the review text that is already collected but not yet analysed, applying language models to extract sentiment and topics and to explain rating differences qualitatively. Periodic re-acquisition would add a temporal dimension and allow the study of rating dynamics, while broadening the geographic scope beyond Milan would test whether the observed platform biases generalize to other markets.

References

[1] E. Zero, S. Ragusini, and V. Yelisieiev. *Data Management Project: Milan Restaurant Ratings Across Online Platforms*. Project repository, source code, report sources, Docker infrastructure, and generated artifacts. https://github.com/Laimon99/data-management-project.

[2] Google Maps Platform. *Places API Documentation*. https://developers.google.com/maps/documentation/places/web-service.

[3] Tripadvisor. Restaurant platform used as a data source. https://www.tripadvisor.com/.

[4] TheFork. Restaurant reservation and review platform used as a data source. https://www.thefork.com/.

[5] OpenStreetMap and Nominatim. Geographic data and geocoding service used for spatial enrichment. https://www.openstreetmap.org/; https://nominatim.org/.

[6] MongoDB. *MongoDB Documentation*. https://www.mongodb.com/docs/.

[7] ClickHouse. *ClickHouse Documentation*. https://clickhouse.com/docs.

[8] Docker. *Docker Documentation*. https://docs.docker.com/.

[9] Python Software Foundation and Astral. *Python* and *uv* package manager. https://www.python.org/; https://docs.astral.sh/uv/.

[10] Microsoft. *Playwright for Python*. https://playwright.dev/python/.

[11] RapidFuzz and GeoPy. Libraries used for fuzzy matching and geocoding. https://rapidfuzz.github.io/RapidFuzz/; https://geopy.readthedocs.io/.

[12] PyMongo and ClickHouse Connect. Python database drivers used by the MongoDB and ClickHouse layers. https://pymongo.readthedocs.io/; https://clickhouse.com/docs/integrations/python.

[13] pandas, SciPy, and Plotly. Libraries used for analysis and visualization. https://pandas.pydata.org/; https://scipy.org/; https://plotly.com/python/.

[14] Pydantic. Data validation and settings management. https://docs.pydantic.dev/.

[15] E. Rahm and P. A. Bernstein. A survey of approaches to automatic schema matching. *The VLDB Journal*, 10(4):334–350, 2001.

[16] P. Christen. *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Springer, 2012.

[17] C. Batini and M. Scannapieco. *Data and Information Quality: Dimensions, Principles and Techniques*. Springer, 2016.

Appendices

The report summarizes the main project pipeline, integration approach, and analytical findings. The full supporting assessment reports were produced as separate project artifacts and are available in the GitHub repository.

A Pre-Integration Data Quality Assessment

The pre-integration quality assessment was completed as a standalone report. It documents the source-level quality checks, metric definitions, field coverage tables, anomaly profiles, and improvement actions used to support the summary in the main text.
Full detailed report 1: Pre-Integration Data Quality Assessment.

B Post-Integration Entity-Resolution and Enrichment Assessment

The post-integration assessment was completed as a standalone report. It documents the entity-resolution evaluation, one-to-one link survival checks, geocoding error analysis, and enrichment-quality evidence used to support the integration discussion in the main text.
Full detailed report 2: Post-Integration Entity-Resolution and Enrichment Assessment.

The same repository also contains the source code, notebooks, Docker configuration, report sources, generated tables, and visualization artifacts used for reproducibility [1].